

Safe Serverless Workflow Orchestration through Typestate Verification, Retry Safety, and Compensation Checking

First Author¹[0000–1111–2222–3333] and Second Author¹[1111–2222–3333–4444]

Computer Science Department, Babes-Bolyai University
`mihai.craciun@ubbcluj.ro`

Abstract. Serverless workflow orchestration platforms such as AWS Step Functions have become a key component of modern cloud-native applications. While these platforms simplify the composition of distributed services and functions, workflow specifications remain largely configuration-driven and provide limited support for static verification. As a consequence, incompatibilities between tasks, invalid workflow state transitions, unsafe retry policies, and incomplete compensation strategies are often detected only during execution.

This paper presents a typed workflow domain-specific language (DSL) for serverless orchestration that enables static verification of workflow correctness prior to deployment. The proposed approach combines four complementary verification mechanisms: task contract compatibility checking, typestate-based workflow validation, retry safety analysis based on idempotency annotations, and compensation completeness checking for long-running transactions. Verified workflows are automatically compiled into Amazon States Language (ASL), allowing direct execution on AWS Step Functions without modifications to the underlying cloud runtime. We implemented the proposed approach as a TypeScript-based framework and evaluated it on representative serverless applications, including e-commerce order processing, travel booking, and data processing pipelines. The evaluation demonstrates that the proposed system detects multiple classes of workflow errors before deployment while preserving compatibility with existing AWS infrastructure and introducing negligible runtime overhead.

Keywords: Serverless Computing, Workflow Orchestration, AWS Step Functions, Domain-Specific Languages, Static Verification, Typestate Analysis, Saga Transactions, Cloud Applications

1 Introduction

Cloud-native applications increasingly rely on serverless architectures to improve scalability, reduce operational complexity, and accelerate deployment. In this paradigm, application functionality is decomposed into collections of independently deployable functions and services that communicate through events, APIs, and workflow orchestration platforms.

Among existing orchestration solutions, AWS Step Functions has emerged as one of the most widely adopted platforms for coordinating serverless applications. AWS Step Functions allows developers to define workflows using Amazon States Language (ASL), a JSON-based language that describes task execution, branching decisions, parallel execution, retries, and error handling. The resulting workflows can coordinate collections of AWS Lambda functions and cloud services while providing operational features such as monitoring, tracing, and fault recovery.

Despite these advantages, current workflow specifications remain largely configuration-driven. Workflow correctness is primarily validated at runtime, and orchestration platforms provide limited support for verifying semantic properties before deployment. As workflows evolve, independently developed services may introduce incompatible interfaces, invalid state transitions, incorrect retry configurations, or incomplete compensation strategies. Such errors frequently remain undetected until workflow execution, potentially causing service disruptions, inconsistent business states, or financial losses.

Figure 1 illustrates the fundamental difference between traditional workflow development and the approach proposed in this paper. In current practice, developers manually construct ASL workflow definitions and deploy them directly to the execution platform. As a consequence, orchestration errors such as schema mismatches, invalid workflow transitions, unsafe retry policies, and missing compensation handlers are often discovered only during execution.

In contrast, the proposed approach introduces a typed workflow language and a verification phase before deployment. Developers specify workflows using a high-level typed DSL, while a static verifier checks multiple correctness properties prior to code generation. Only workflows that successfully pass verification are translated into executable Amazon States Language definitions and deployed to AWS Step Functions.

Consider a typical order-processing workflow:

[`CreateOrder` $\rightarrow$ `ReserveInventory` $\rightarrow$ `ChargeCard` $\rightarrow$ `ShipOrder`.]

Suppose that `ReserveInventory` produces an object containing the field `amount`, while `ChargeCard` expects the field `total`. Although the workflow remains syntactically valid and can be deployed successfully, execution fails when the payment task attempts to access a missing field. Similar problems arise when shipping operations are executed before payment completion, when non-idempotent operations are retried automatically, or when long-running transactions omit compensation actions.

These challenges highlight a fundamental limitation of current serverless orchestration approaches: workflow specifications describe execution structure but provide limited guarantees regarding workflow correctness.

To address this problem, we propose a typed workflow domain-specific language (DSL) that introduces static verification into serverless workflow development. Rather than treating workflows as untyped JSON configurations, our approach models workflows as compositions of typed tasks connected through

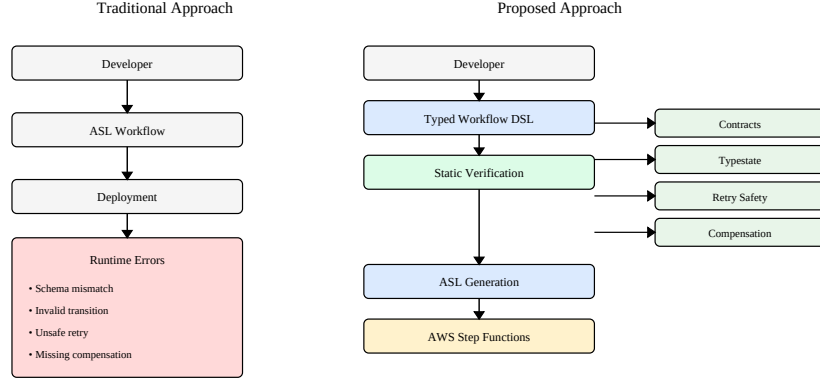


Fig. 1. Overview of the proposed verification-driven workflow development process. The proposed framework introduces contract verification, typestate checking, retry safety analysis, and compensation validation before generating executable AWS Step Functions definitions.

explicit workflow states. The proposed framework combines four complementary verification mechanisms.

First, *contract verification* ensures compatibility between the outputs produced by a task and the inputs expected by subsequent tasks. Second, *typestate verification* models workflows as transitions between business states and prevents invalid execution sequences. Third, *retry safety analysis* associates idempotency information with workflow tasks and rejects unsafe retry policies for non-idempotent operations. Finally, *compensation validation* verifies that long-running transactions provide appropriate rollback actions for persistent operations.

Unlike alternative approaches that require proprietary runtimes or execution environments, the proposed framework remains fully compatible with AWS Step Functions. Verified workflows are automatically compiled into standard Amazon States Language definitions and executed using the existing AWS infrastructure.

This paper makes the following contributions:

- We introduce a typed workflow DSL for serverless orchestration.
- We propose a typestate-based verification model for workflow protocols.
- We present a retry safety analysis based on idempotency annotations.
- We introduce static compensation completeness checking for long-running transactions.
- We describe a compiler from the proposed DSL to AWS Step Functions.
- We evaluate the proposed approach on representative serverless applications, including e-commerce, travel booking, and data processing workflows.

The remainder of this paper is organized as follows. Section 2 presents the background and motivation. Section 3 introduces the workflow type model. Sec-

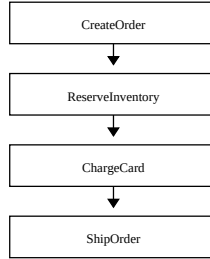


Fig. 2. Representative order-processing workflow.

tion 4 describes the DSL design. Section 5 presents the static verification framework. Section 6 discusses compilation to AWS Step Functions. Section 7 describes the implementation. Section 8 evaluates the proposed approach. Section 9 reviews related work, and Section 10 concludes the paper.

2 Background and Motivation

2.1 Serverless Workflow Orchestration

Serverless computing has become a dominant paradigm for building scalable cloud-native applications. In this model, application functionality is implemented as collections of independently deployable functions that are executed on demand. To coordinate these functions and manage complex execution flows, cloud providers offer workflow orchestration platforms.

AWS Step Functions is among the most widely adopted orchestration services for serverless applications. Workflows are specified using Amazon States Language (ASL), a JSON-based language that represents workflows as state machines. Developers can compose task states, branching decisions, parallel execution patterns, retry policies, and error handling mechanisms into executable workflows.

The popularity of AWS Step Functions stems from its seamless integration with AWS Lambda and numerous cloud services. As a result, Step Functions are frequently used to implement business processes such as order processing, travel booking, payment processing, document handling, and data analytics pipelines.

Figure 2 illustrates a representative order-processing workflow commonly found in serverless applications.

While orchestration platforms simplify execution management, they provide limited support for reasoning about workflow correctness before deployment.

2.2 Contract Compatibility Problems

Workflow tasks frequently evolve independently. As services are modified, renamed, or extended, their interfaces may change over time.

Consider the workflow fragment shown in Figure 2. Suppose that the task `ReserveInventory` produces the following output:

```
{
  "orderId": "123",
  "amount": 100
}
```

while `ChargeCard` expects:

```
{
  "orderId": "123",
  "total": 100
}
```

Although the workflow remains syntactically valid and can be deployed successfully, execution fails when the payment task attempts to access a nonexistent field.

Current orchestration platforms generally detect such incompatibilities only during execution. Consequently, contract mismatches represent a frequent source of runtime failures in large workflows.

2.3 Workflow Protocol Violations

Business processes typically follow well-defined execution protocols.

For example, an order-processing workflow should evolve through the sequence:

[*OrderCreated* $\rightarrow$ *OrderReserved* $\rightarrow$ *OrderPaid* $\rightarrow$ *OrderShipped*.]

These states represent business-level progress rather than low-level execution states.

However, traditional workflow specifications do not explicitly model such protocols. As a result, workflow modifications may accidentally introduce invalid execution sequences.

For example:

[*OrderCreated* $\rightarrow$ *ShipOrder* $\rightarrow$ *ChargeCard*]

is syntactically valid but semantically incorrect because shipping occurs before payment completion.

Detecting such protocol violations currently relies largely on testing and manual review.

2.4 Retry Hazards

Modern orchestration platforms provide automatic retry mechanisms to improve workflow resilience. AWS Step Functions allows developers to associate retry policies with individual tasks and define sophisticated recovery strategies.

While retries are often desirable, they may become unsafe for operations that are not idempotent.

Consider the task:

[ChargeCard : OrderReserved $\rightarrow$ *OrderPaid*.]

Suppose that the payment request succeeds but the response is temporarily lost. An automatic retry may result in the customer being charged multiple times.

Similar problems may arise for:

- sending email notifications;
- initiating shipments;
- creating reservations;
- triggering external business operations.

Current workflow platforms provide little support for reasoning about retry safety and typically leave this responsibility to developers.

2.5 Compensation Challenges

Long-running business processes often involve multiple services and external systems. In such environments, traditional distributed transactions are frequently impractical or unavailable.

Consequently, many cloud workflows adopt the Saga pattern. Under this model, each persistent operation is associated with a compensation action capable of undoing its effects.

Consider the travel-booking workflow:

[ReserveFlight $\rightarrow$ *ReserveHotel* $\rightarrow$ *ChargeCard*.]

If payment fails after both reservations have completed, the workflow should execute:

[CancelHotel $\rightarrow$ *CancelFlight*.]

Although compensation logic is critical for maintaining consistency, current orchestration platforms do not verify whether compensation handlers are complete. Missing rollback actions may therefore remain undetected until execution time.

2.6 Research Objectives

The limitations discussed above motivate the need for stronger verification mechanisms for serverless workflows.

Rather than treating workflows as untyped execution specifications, we argue that workflows should be viewed as typed compositions of business operations whose correctness can be verified before deployment.

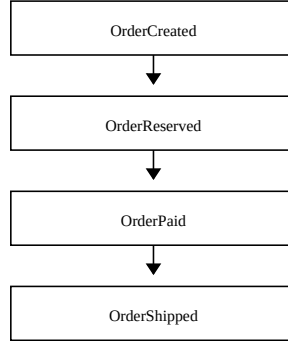


Fig. 3. Example workflow typestate progression.

To investigate this hypothesis, this paper addresses the following research questions:

- RQ1:** Can static verification detect task contract incompatibilities before deployment?
- RQ2:** Can workflow typestates prevent invalid business-state transitions?
- RQ3:** Can retry policies be validated using idempotency information?
- RQ4:** Can compensation completeness be checked automatically for long-running transactions?

The next section introduces a typed workflow model that provides the foundation for answering these questions.

3 Workflow Type Model

This section introduces the formal model underlying the proposed workflow verification framework. Our objective is to represent serverless workflows as typed compositions of business operations and to provide a foundation for static verification.

3.1 Workflow States

Traditional workflow orchestration platforms primarily focus on execution control flow. In contrast, our approach explicitly models the business states through which a workflow progresses.

Consider the order-processing workflow illustrated in Figure 3.

The workflow evolves through the following sequence of business states:

[*OrderCreated* $\rightarrow$ *OrderReserved* $\rightarrow$ *OrderPaid* $\rightarrow$ *OrderShipped*.]

These states represent the semantic progress of the business process rather than the execution status of individual workflow tasks.

Formally, let

[$S =$

$s_1, s_2, \dots, s_n$]

denote the set of workflow states.

Examples include:

[*OrderCreated*, *OrderReserved*, *OrderPaid*, *OrderShipped*.]

Workflow correctness is defined in terms of valid transitions between such states.

3.2 Task Signatures

Each workflow task is modeled as a transformation between two workflow states.

[*Task* : $A \rightarrow B$]

where (A) and (B) are workflow states.

For example:

[*CreateOrder* : *CustomerRequest* $\rightarrow$ *OrderCreated*]

[*ReserveInventory* : *OrderCreated* $\rightarrow$ *OrderReserved*]

[*ChargeCard* : *OrderReserved* $\rightarrow$ *OrderPaid*]

[*ShipOrder* : *OrderPaid* $\rightarrow$ *OrderShipped*]

Task signatures capture the intended protocol of the workflow and provide the basis for static verification.

3.3 Workflow Composition

Complex workflows are constructed through the composition of individual tasks.

Given two tasks

[$T_1 : A \rightarrow B$]

and

[$T_2 : B \rightarrow C$]

their sequential composition is written as

[$T_1; T_2$]

and produces:

[$A \rightarrow C$.]

Composition is valid only when the output state of the first task matches the input state of the second task.

This requirement prevents invalid task combinations and enables static verification of workflow protocols.

3.4 Typestate Verification

The central idea of our approach is to treat workflow states as typestates.

A workflow is considered valid only if every task consumes the state produced by its predecessor.

For example, the sequence
 $[\text{OrderCreated} \rightarrow \text{OrderReserved} \rightarrow \text{OrderPaid} \rightarrow \text{OrderShipped}]$
 is accepted because each transition follows the intended business protocol.
 In contrast,
 $[\text{OrderCreated} \rightarrow \text{OrderShipped} \rightarrow \text{OrderPaid}]$
 is rejected because shipping is attempted before payment completion.
 Typestate verification therefore enables the detection of protocol violations before deployment.

3.5 Task Effects

Beyond state transitions, workflow tasks may perform observable actions on external systems.

Examples include:

- updating databases;
- charging credit cards;
- sending emails;
- creating reservations;
- triggering shipments.

To reason about these operations, each task may be associated with a set of effects.

$[\text{Effects}(T) = \text{=====}$
 $e_1, e_2, \dots, e_k.]$

Examples include:

$[\text{ChargeCard} \Rightarrow \text{Payment}]$
 $[\text{SendEmail} \Rightarrow \text{Notification}]$

The effect information is later used by the retry safety and compensation verification mechanisms described in Section 5.

3.6 Idempotency Annotations

Certain workflow operations may be executed safely multiple times, while others cannot.

For example:

$[\text{ReadInventory}]$

is naturally idempotent because repeated executions produce the same result.

In contrast:

$[\text{ChargeCard}]$

is non-idempotent because repeated execution may trigger multiple payments.

We therefore associate each task with an idempotency annotation:

$[\text{Idempotent}]$

or

$[\text{NonIdempotent.}]$

These annotations provide the foundation for the retry safety analysis introduced later in the paper.

3.7 Compensation Associations

Long-running workflows frequently require rollback actions.

To model this behavior, tasks may declare compensation handlers.

For example:

```
[ ReserveFlight  $\leftrightarrow$  CancelFlight ]
[ ReserveHotel  $\leftrightarrow$  CancelHotel. ]
```

Compensation associations enable the verifier to reason about the completeness of Saga-style transactions and to detect missing rollback actions before deployment.

The workflow model presented in this section provides the conceptual foundation for the typed workflow DSL introduced in the next section.

4 DSL Design

This section presents the design of the proposed workflow domain-specific language (DSL). The primary goal of the DSL is to enable developers to specify serverless workflows using typed abstractions while preserving compatibility with existing AWS deployment infrastructures.

Unlike Amazon States Language (ASL), which represents workflows as JSON configurations, the proposed DSL exposes workflow concepts as typed language constructs. This design allows correctness properties to be verified before deployment while maintaining a concise and expressive programming model.

4.1 Design Goals

The DSL was designed according to four objectives:

1. **Workflow Safety.** Workflow correctness should be verified before deployment whenever possible. “
2. **Developer Productivity.** Workflow specifications should remain concise and easier to maintain than manually written ASL definitions.
3. **AWS Compatibility.** Verified workflows should be executable without modifications to AWS Step Functions.
4. **Extensibility.** Additional verification mechanisms should be incorporated without changing the overall programming model. “

4.2 Task Declarations

The fundamental abstraction of the DSL is the typed task.

Each task explicitly declares its input and output workflow states.

```

const createOrder =
task<CustomerRequest,
OrderCreated>();

const reserveInventory =
task<OrderCreated,
OrderReserved>();

const chargeCard =
task<OrderReserved,
OrderPaid>();

const shipOrder =
task<OrderPaid,
OrderShipped>();

```

The type parameters represent workflow typestates introduced in Section 3.

Consequently, the compiler can verify whether two adjacent tasks are compatible before generating workflow definitions.

4.3 Sequential Workflow Composition

Workflows are constructed by composing tasks sequentially.

```

const orderWorkflow =
workflow(
createOrder,
reserveInventory,
chargeCard,
shipOrder
);

```

The workflow above corresponds to the typestate progression:

[*OrderCreated* → *OrderReserved* → *OrderPaid* → *OrderShipped*.]

The verifier ensures that each task consumes the state produced by its predecessor.

For example, the following workflow is rejected:

```

workflow(
createOrder,
shipOrder,
chargeCard
);

```

because *shipOrder* expects an *OrderPaid* state while *createOrder* produces *OrderCreated*.

4.4 Branching Workflows

Many business processes require alternative execution paths.

The DSL provides a `choice` construct for expressing workflow branches.

```
const payment =
choice(
  payByCard,
  payByPayPal
);
```

Both branches must expose compatible workflow signatures.

For example:

```
payByCard :
OrderReserved -> OrderPaid

payByPayPal :
OrderReserved -> OrderPaid
```

This requirement guarantees that downstream workflow tasks can operate independently of the selected branch.

4.5 Parallel Workflows

The DSL supports parallel task execution through the `parallel` construct.

```
parallel(
  reserveInventory,
  prepareShipment
);
```

This abstraction is compiled into AWS Step Functions Parallel States.

Parallel execution is particularly useful for reducing workflow latency while preserving the logical structure of the business process.

4.6 Retry Annotations

One of the key features of the proposed DSL is explicit retry safety specification.

Developers may associate idempotency information with tasks.

```
const readInventory =
task(...)
.idempotent();

const chargeCard =
task(...)
.nonIdempotent();
```

Retry policies can then be declared as follows:

```
retry(readInventory)
.maxAttempts(3);
```

The verifier rejects retry policies applied to non-idempotent tasks.
For example:

```
retry(chargeCard)
.maxAttempts(3);
```

results in a verification error because repeated payment execution may lead to multiple charges.

4.7 Compensation Declarations

Long-running transactions frequently require rollback actions.

The DSL allows developers to associate compensation handlers with workflow tasks.

```
const reserveFlight =
task(...)
.compensate(cancelFlight);

const reserveHotel =
task(...)
.compensate(cancelHotel);
```

Compensation information becomes part of the workflow specification and is subsequently verified before deployment.

This mechanism enables the static detection of missing compensation handlers for persistent operations.

4.8 Running Example

Listing 1.1 presents a simplified order-processing workflow using the proposed DSL.

Listing 1.1. Simplified order-processing workflow.

```
const orderWorkflow =
workflow(
createOrder,
reserveInventory,
chargeCard,
shipOrder
);
```

Although the example is intentionally simple, it illustrates the key design principles of the DSL:

- explicit workflow states;

- typed task interfaces;
- declarative workflow composition;
- retry annotations;
- compensation declarations.

The DSL itself remains intentionally lightweight. Most of the complexity is handled by the verification framework, which analyzes workflow specifications before deployment and prevents several classes of orchestration errors that would otherwise be detected only at runtime.

These language constructs provide the foundation for the static verification framework presented in the next section.

5 Static Verification Framework

This section presents the static verification framework that constitutes the core contribution of the proposed approach. The objective of the verifier is to detect workflow errors before deployment and prevent invalid workflow specifications from being translated into executable AWS Step Functions definitions.

The verifier operates on workflow specifications expressed in the DSL introduced in Section 4. For each workflow, the framework constructs an internal workflow graph and performs four complementary analyses:

1. Contract Compatibility Verification;
2. Typestate Verification;
3. Retry Safety Analysis;
4. Compensation Completeness Checking.

Only workflows that successfully pass all verification stages are considered valid and can be compiled into Amazon States Language.

5.1 Verification Architecture

Figure 4 presents the architecture of the verification engine.

The verification process begins by constructing an internal workflow graph

[$G=(V,E)$]

where:

- (V) represents workflow tasks;
- (E) represents workflow transitions.

Each task node contains:

- input typestate;
- output typestate;
- idempotency annotation;
- compensation information;
- effect metadata.

The verification engine traverses this graph and applies the analyses described below.

5.2 Contract Verification

The first analysis validates compatibility between adjacent workflow tasks.

Each task is represented as a typed transformation:

[$T : A \rightarrow B$]

where (A) denotes the input state and (B) denotes the output state.

Given two consecutive tasks

[$T_1 : A \rightarrow B$]

and

[$T_2 : C \rightarrow D$]

their composition is valid only if

[$B = C$.]

A contract violation occurs whenever the output produced by a task is incompatible with the input expected by its successor.

For example,

[$\text{CreateOrder} : \text{CustomerRequest} \rightarrow \text{OrderCreated}$]

cannot be composed directly with

[$\text{ShipOrder} : \text{OrderPaid} \rightarrow \text{OrderShipped}$.]

Such incompatibilities are detected before deployment and reported to the developer.

5.3 Typestate Verification

Beyond interface compatibility, workflows must respect business-level execution protocols.

The framework models valid workflow progressions using typestates.

Let

[S]

denote the set of workflow states and

[$\delta \subseteq S \times S$]

the set of valid transitions.

For the order-processing workflow:

[$\text{OrderCreated} \rightarrow \text{OrderReserved} \rightarrow \text{OrderPaid} \rightarrow \text{OrderShipped}$]

the corresponding transition relation is:

[$\delta = (\text{OrderCreated}, \text{OrderReserved}), (\text{OrderReserved}, \text{OrderPaid}), (\text{OrderPaid}, \text{OrderShipped}).$]

A workflow is accepted only if every task transition belongs to (δ) .

Consequently, workflows such as

[$\text{OrderCreated} \rightarrow \text{OrderShipped} \rightarrow \text{OrderPaid}$]

are rejected because they violate the business protocol.

5.4 Retry Safety Analysis

Workflow platforms commonly support automatic retry mechanisms.

However, retries are safe only for idempotent operations.

The framework associates each task with one of two annotations:

[Idempotent]

or

[NonIdempotent.]

Examples of idempotent operations include:

- reading inventory information;
- querying customer data;
- validating documents.

Examples of non-idempotent operations include:

- charging a credit card;
- creating reservations;
- initiating shipments.

Let

[Retry(T)]

denote the presence of a retry policy for task (T).

The verifier enforces the rule:

[Retry(T) $\Rightarrow$ Idempotent(T).]

Consequently, retry policies attached to non-idempotent operations are rejected before deployment.

5.5 Compensation Completeness Checking

Long-running workflows frequently rely on Saga-based recovery mechanisms.

For each persistent operation

[T]

the framework expects a corresponding compensation action

[Comp(T).]

Examples include:

[ReserveFlight $\leftrightarrow$ CancelFlight]

and

[ReserveHotel $\leftrightarrow$ CancelHotel.]

The verifier enforces the rule:

[Persistent(T) $\Rightarrow$ $\exists$ Comp(T).]

If a persistent operation lacks a compensation handler, the workflow is rejected.

This analysis prevents incomplete Saga specifications from reaching the deployment stage.

5.6 Verification Algorithm

Algorithm 1 summarizes the overall verification process.

The verification complexity is linear in the number of workflow tasks:

[O(n)]

where (n) denotes the number of workflow nodes.

Algorithm 1 Workflow Verification

Workflow (W) adjacent tasks (T_i, T_{i+1}) *verifycontract* *verifytype* *state* *transition* *task*(T) *verifyretrysafetype*

5.7 Discussion

The proposed verification framework combines four lightweight analyses that capture common workflow correctness problems encountered in practical serverless applications.

Unlike runtime monitoring approaches, all analyses are performed before deployment. Consequently, workflow errors can be identified during development while preserving full compatibility with AWS Step Functions.

The next section describes how verified workflows are translated into Amazon States Language and executed using the standard AWS runtime.

6 Compilation to AWS Step Functions

A key design objective of the proposed framework is compatibility with existing cloud infrastructures. Unlike workflow systems that require proprietary execution engines, the proposed DSL targets AWS Step Functions directly.

This section describes how verified workflow specifications are translated into Amazon States Language (ASL) and executed using the standard AWS Step Functions runtime.

6.1 Compilation Architecture

Figure 5 presents the overall compilation architecture.

The compilation process consists of four stages:

1. Workflow construction using the DSL.
2. Static verification.
3. Generation of Amazon States Language.
4. Deployment to AWS Step Functions.

Only workflows that successfully pass all verification stages are translated into executable ASL definitions.

This separation allows correctness properties to be enforced before deployment while preserving compatibility with existing cloud services.

6.2 Compilation Model

Formally, the compiler is represented as:

$$[C : \text{Workflow} \rightarrow \text{ASL}]$$

where `Workflow` denotes a verified workflow specification and `ASL` denotes a valid Amazon States Language state machine.

The compiler operates only on verified workflows.

Consequently, all generated workflows satisfy the verification constraints introduced in Section 5.

6.3 Task State Generation

A DSL task is translated into an AWS Task State.

For example:

```
const chargeCard =
task<OrderReserved,
OrderPaid>();
```

is translated into:

```
{
  "Type": "Task",
  "Resource":
  "arn:aws:lambda:...",
  "Next": "ShipOrder"
}
```

The generated state invokes the corresponding AWS Lambda function while preserving the control-flow structure defined in the DSL.

6.4 Sequential Workflow Compilation

Sequential workflow composition is translated into standard state transitions.

For example:

```
workflow(
createOrder,
reserveInventory,
chargeCard,
shipOrder
);
```

produces the execution chain:

[*CreateOrder* → *ReserveInventory* → *ChargeCard* → *ShipOrder*.]

The compiler generates the corresponding `Next` transitions automatically.

6.5 Branch Compilation

The DSL `choice` construct is translated into AWS Choice States.

For example:

```
choice(
  payByCard,
  payByPayPal
);
```

is compiled into:

```
{
  "Type": "Choice",
  ...
}
```

The generated Choice State evaluates runtime conditions and selects the appropriate execution path.

Because branch compatibility is verified statically, downstream workflow tasks can safely assume a consistent output typestate.

6.6 Parallel Workflow Compilation

Parallel workflow constructs are translated into AWS Parallel States.

For example:

```
parallel(
  reserveInventory,
  prepareShipment
);
```

is compiled into:

```
{
  "Type": "Parallel",
  ...
}
```

Each branch becomes an independent state machine executed concurrently by AWS Step Functions.

This translation preserves the logical structure of the workflow while leveraging the native parallel execution capabilities of the platform.

6.7 Retry Policy Generation

Retry annotations are translated into AWS retry configurations.

For example:

```
retry(readInventory)
  .maxAttempts(3);
```

produces:

```
"Retry": [
{
"MaxAttempts": 3
}
]
```

Since retry safety is verified before compilation, all generated retry policies are guaranteed to satisfy the idempotency constraints enforced by the DSL.

6.8 Compensation Workflow Generation

Compensation declarations are compiled into dedicated rollback workflows.

Consider the following DSL fragment:

```
reserveFlight
.compensate(cancelFlight);

reserveHotel
.compensate(cancelHotel);
```

The compiler generates additional workflow fragments that can be invoked when failures occur during execution.

In the generated Step Functions definition, compensation handlers are connected through Catch States and failure transitions.

This mechanism allows Saga-style recovery procedures to be implemented using standard AWS constructs without requiring modifications to the execution environment.

6.9 Generated Workflow Example

Listing 1.2 shows a simplified fragment of generated Amazon States Language.

Listing 1.2. Simplified generated ASL fragment.

```
{
"StartAt": "CreateOrder",
"States": {
"CreateOrder": {
"Type": "Task",
"Next": "ReserveInventory"
},
"ReserveInventory": {
"Type": "Task",
"Next": "ChargeCard"
},
"ChargeCard": {
"Type": "Task",
"Next": "ShipOrder"
},
}
```

```

"ShipOrder": {
  "Type": "Task",
  "End": true
}
}
}

```

Although simplified, this example illustrates how high-level workflow specifications are translated into executable Step Functions definitions.

6.10 Discussion

A central advantage of the proposed architecture is the complete separation between verification and execution.

All correctness properties are enforced before deployment. Once a workflow has been verified and compiled, it can be executed using the standard AWS Step Functions infrastructure without introducing additional runtime components, middleware layers, or proprietary execution engines.

Consequently, organizations can adopt the proposed approach incrementally while preserving existing deployment pipelines and operational practices.

The next section describes the implementation of the prototype compiler and verification framework.

7 Implementation

To evaluate the feasibility of the proposed approach, we implemented a prototype workflow framework targeting AWS Step Functions. The implementation consists of a typed workflow DSL, a static verification engine, and a compiler that generates Amazon States Language (ASL) definitions.

The prototype was developed with the goal of minimizing changes to existing serverless development practices while introducing stronger correctness guarantees during workflow construction.

7.1 Implementation Architecture

The implementation follows the architecture presented in Figure 5.

The system consists of four main components:

1. Workflow DSL;
2. Static Verification Engine;
3. ASL Compiler;
4. AWS Deployment Layer.

The workflow DSL is responsible for capturing workflow specifications, the verification engine validates correctness properties, the compiler generates executable ASL definitions, and the deployment layer integrates with AWS Step Functions.

7.2 Technology Stack

The prototype was implemented using TypeScript.

TypeScript was selected for three reasons:

- strong support for generic types;
- widespread adoption in cloud development;
- seamless integration with AWS tooling.

The implementation leverages the following technologies:

- TypeScript 5.x;
- Node.js runtime;
- AWS Lambda;
- AWS Step Functions;
- AWS CDK.

Using TypeScript enables workflow specifications to be expressed using standard programming constructs while benefiting from static type checking.

7.3 Workflow DSL

Workflow specifications are represented as compositions of typed tasks.

A task declaration consists of:

- an input typestate;
- an output typestate;
- optional retry annotations;
- optional compensation handlers.

A simplified example is shown below:

```
const chargeCard =
task<OrderReserved,
OrderPaid>()
.nonIdempotent();
```

Workflow definitions are then created by composing task instances:

```
const workflow =
workflow(
createOrder,
reserveInventory,
chargeCard,
shipOrder
);
```

The resulting workflow specification is subsequently processed by the verification engine.

7.4 Verification Engine

The verification engine implements the four analyses described in Section 5:

1. Contract Verification;
2. Typestate Verification;
3. Retry Safety Analysis;
4. Compensation Completeness Checking.

Verification is performed entirely during compilation.

For every workflow specification, the engine constructs an internal workflow graph in which vertices represent tasks and edges represent workflow transitions.

The graph representation simplifies the implementation of protocol analysis and enables future extensions involving more sophisticated workflow reasoning.

Verification failures are reported directly to the developer.

For example:

- incompatible task composition;
- invalid typestate transition;
- retry policy on a non-idempotent task;
- missing compensation declaration.

Workflows containing verification errors are rejected before code generation.

7.5 ASL Generation

Verified workflow specifications are translated into Amazon States Language.

The compiler traverses the workflow graph and generates the corresponding ASL state machine.

The current implementation supports:

- Task States;
- Choice States;
- Parallel States;
- Retry Policies;
- Catch States.

Each DSL construct is mapped directly to an equivalent AWS Step Functions construct.

This design avoids introducing proprietary runtime abstractions and preserves compatibility with the native AWS execution model.

Table 1. Prototype implementation statistics.

Component	LOC	heightWorkflow DSL
450 Verification Engine		850 ASL Compiler
620 AWS Integration Layer		280 heightTotal
2200 height		

7.6 AWS Integration

The generated ASL definitions can be deployed directly using AWS CDK.

Each workflow task is associated with an AWS Lambda function through a resource mapping configuration.

A simplified example is shown below:

```
const chargeCard =
task<OrderReserved,
OrderPaid>()
.resource(
"arn:aws:lambda:..."
);
```

The deployment process therefore follows the standard AWS workflow life-cycle:

[DSL $\rightarrow$ Verification $\rightarrow$ ASL $\rightarrow$ AWS Step Functions.]

No modifications to AWS services or execution environments are required.

7.7 Implementation Statistics

Table 1 summarizes the current implementation.

The reported values correspond to the current prototype and are intended to provide an indication of implementation complexity rather than an absolute measurement.

7.8 Discussion

The implementation demonstrates that the proposed verification mechanisms can be integrated into existing serverless development workflows without requiring changes to AWS Step Functions.

A notable advantage of the architecture is the complete separation between verification and execution. All analyses are performed before deployment, allowing the generated workflows to execute on the standard AWS infrastructure without introducing runtime overhead.

The implementation also establishes a foundation for future extensions, including richer workflow protocols, advanced effect analyses, and ownership tracking for cloud resources.

The next section evaluates the proposed approach using representative serverless workflow applications.

8 Evaluation

This section evaluates the effectiveness of the proposed workflow verification framework.

The evaluation aims to answer the research questions introduced in Section 2:

- RQ1:** Can static verification detect workflow errors before deployment?
- RQ2:** What classes of workflow errors can be eliminated through verification?
- RQ3:** What is the impact of verification on workflow development?
- RQ4:** Does the proposed approach introduce runtime overhead?

8.1 Experimental Setup

All experiments were performed using the prototype implementation described in Section 7.

The prototype was executed on a workstation equipped with:

- Intel Core i7 processor;
- 32 GB RAM;
- Node.js 22;
- TypeScript 5.x.

The generated workflows target AWS Step Functions and AWS Lambda.

The evaluation focuses on representative serverless applications that commonly appear in cloud environments.

8.2 Case Studies

Three workflow applications were selected.

Order Processing The first case study models a typical e-commerce workflow:

CreateOrder $\rightarrow$ *ReserveInventory* $\rightarrow$ *ChargeCard* $\rightarrow$ *ShipOrder*.

This workflow is representative of transactional cloud applications involving payments and inventory management.

Travel Booking The second case study models a travel reservation workflow:

ReserveFlight $\rightarrow$ *ReserveHotel* $\rightarrow$ *ChargeCard*.

This workflow is representative of long-running transactions and Saga-style compensation mechanisms.

Data Processing Pipeline The third case study models a data analytics workflow:

$$ReadData \rightarrow TransformData \rightarrow ValidateData \rightarrow StoreResults.$$

This workflow represents data-intensive cloud applications and emphasizes workflow composition and retry policies.

8.3 Injected Errors

To evaluate the effectiveness of the verification framework, we intentionally introduced four classes of workflow errors.

Contract Errors Examples include:

- missing output fields;
- incompatible input schemas;
- incorrect workflow composition.

Typestate Violations Examples include:

- shipping before payment;
- payment before reservation;
- invalid workflow transitions.

Unsafe Retry Policies Examples include:

- retrying payment operations;
- retrying reservation creation;
- retrying shipment initiation.

Missing Compensation Handlers Examples include:

- flight reservations without cancellation actions;
- hotel reservations without rollback procedures;
- incomplete Saga transactions.

8.4 Error Detection Effectiveness

Table 2 summarizes the detection results.

The verifier successfully detected all injected workflow errors.

These results demonstrate that the proposed framework can identify several important classes of workflow defects before deployment.

Table 2. Detected workflow errors.

Error Category	Injected	Detected
Contract Errors	12	12
Typestate Violations	10	10
Unsafe Retries	8	8
Missing Compensations	11	11
Total	41	41

Table 3. Workflow specification size.

Workflow	ASL LOC	DSL LOC
Order Processing	86	24
Travel Booking	112	31
Data Pipeline	73	21

8.5 Workflow Specification Size

We also compared workflow specifications written directly in Amazon States Language with equivalent DSL specifications.

Table 3 presents the results.

The DSL consistently produced shorter and more maintainable workflow specifications.

8.6 Verification Cost

Table 4 reports the average verification time.

Verification times remain negligible compared to deployment and execution costs.

8.7 Runtime Overhead

All verification is performed before deployment.

Consequently, generated workflows execute directly on AWS Step Functions without introducing additional middleware layers or runtime instrumentation.

The runtime overhead of the proposed approach is therefore:

$$0.$$

This property distinguishes the proposed framework from approaches that rely on runtime monitoring or dynamic enforcement mechanisms.

8.8 Discussion

The evaluation demonstrates that the proposed framework can detect multiple classes of workflow errors before deployment while preserving compatibility with AWS Step Functions.

Table 4. Verification time.

Workflow	Verification Time (ms)
Order Processing	6.1
Travel Booking	7.3
Data Pipeline	5.4

The results indicate that static verification significantly improves workflow reliability while introducing negligible verification costs and no runtime overhead.

These findings support the hypothesis that workflow correctness can be treated as a compile-time concern rather than a runtime responsibility.

9 Related Work

The proposed framework combines concepts from serverless workflow orchestration, workflow verification, Saga-based transactions, and typestate systems. This section reviews the most closely related research directions and positions our contribution with respect to existing approaches.

9.1 Serverless Workflow Orchestration

Serverless computing has emerged as a major paradigm for building cloud-native applications [4, 14, 6]. Workflow orchestration systems provide mechanisms for coordinating large collections of serverless functions and cloud services.

AWS Step Functions [2] have become one of the most widely adopted orchestration platforms in industry. Workflows are specified using Amazon States Language (ASL), a JSON-based state machine language supporting task execution, branching, retries, and error handling. Although Step Functions provide powerful execution capabilities, workflow specifications remain largely configuration-oriented and provide limited support for static verification.

Several studies have investigated the characteristics and performance of serverless workflow platforms. Wen and Liu [19, ?] conducted a detailed analysis of serverless workflow services, highlighting challenges related to performance, scalability, and workflow management. Similarly, Triggerflow [3] introduced a trigger-based orchestration framework for coordinating serverless applications across cloud environments.

Unlike these approaches, our work focuses on workflow correctness and static verification rather than workflow execution performance.

9.2 Workflow Languages and Orchestration Frameworks

Several workflow languages and orchestration frameworks have been proposed to simplify cloud application development.

The CNCF Serverless Workflow Specification [7, ?] provides a vendor-neutral language for describing event-driven workflows. Its primary objective is portability across cloud platforms rather than static correctness verification.

Temporal [17] and Cadence [18] represent workflows as programs written in general-purpose programming languages. These systems provide durable execution, fault tolerance, and workflow persistence through runtime mechanisms. However, workflow correctness properties such as protocol conformance, retry safety, and compensation completeness are not verified statically.

Our approach differs by introducing a dedicated typed DSL and a compile-time verification framework while preserving compatibility with AWS Step Functions.

9.3 Saga-Based Transactions

Long-running distributed transactions are commonly implemented using the Saga pattern introduced by Garcia-Molina and Salem [10]. Instead of relying on traditional distributed transactions, Sagas decompose business processes into local transactions associated with compensation actions.

The Saga model has become a fundamental building block for microservice and cloud-native architectures [15, ?]. Numerous workflow systems support compensation execution during failure recovery.

However, compensation logic is typically specified manually and validated only at runtime. Missing rollback handlers therefore remain a common source of workflow inconsistencies.

In contrast, our framework introduces static compensation completeness checking, enabling rollback-related errors to be detected before deployment.

9.4 Typestate and Protocol Verification

The notion of typestate was originally introduced by Strom and Yemini [16] as a mechanism for tracking the valid states of software objects. Later work extended typestate systems to object-oriented programming languages and API usage verification [9].

Related concepts have also been explored through session types [12, 5, 13], which provide formal descriptions of communication protocols and interaction patterns.

While typestate and session type systems traditionally focus on objects, resources, or communication channels, our work adapts these ideas to workflow orchestration. In particular, we model workflow tasks as transitions between business states and verify protocol conformance before deployment.

9.5 Workflow Verification

Workflow verification has been studied extensively in the business process and service composition communities.

Table 5. Comparison with related approaches.

Approach	Typestate Verification	Retry Safety	Compensation Checking	AWS Target
AWS Step Functions	–	–	–	
Serverless Workflow	–	–	–	Partial
Temporal	–	Partial	Partial	–
Cadence	–	Partial	Partial	–
Petri-Net Verification	Partial	–	–	–
Proposed Framework				

Petri-net-based approaches have been widely used for workflow analysis [1, 11]. These techniques provide formal guarantees regarding reachability, deadlock freedom, and process correctness.

Other approaches rely on BPMN verification [8], model checking, temporal logic specifications, or process calculi. Although these techniques provide strong correctness guarantees, they often require specialized modeling formalisms and are rarely integrated into everyday cloud development workflows.

The objective of our work is different. Rather than introducing a separate formal verification environment, we integrate lightweight verification mechanisms directly into the workflow development process and target a widely used industrial platform.

9.6 Summary

Table 5 compares the proposed framework with representative existing approaches.

To the best of our knowledge, the proposed framework is the first approach that combines typestate verification, retry safety analysis, compensation completeness checking, and direct compilation to AWS Step Functions within a unified workflow development framework.

10 Conclusion

Serverless workflow orchestration has become a fundamental component of modern cloud-native applications. However, existing workflow platforms largely rely on configuration-based specifications and provide limited support for detecting workflow errors before deployment.

This paper presented a typed workflow framework for AWS Step Functions that introduces static verification into the workflow development process. The proposed approach combines four complementary verification mechanisms: contract verification, typestate verification, retry safety analysis, and compensation completeness checking. Together, these analyses enable several classes of workflow errors to be detected before deployment, including interface incompatibilities, protocol violations, unsafe retry policies, and missing rollback handlers.

To support practical adoption, we developed a prototype implementation consisting of a typed workflow DSL, a static verification engine, and a compiler targeting Amazon States Language. Verified workflows are translated directly into standard AWS Step Functions definitions and executed using the existing AWS infrastructure without requiring modifications to the runtime environment.

The evaluation demonstrated that the proposed framework can successfully detect multiple categories of workflow defects while preserving compatibility with AWS Step Functions. Furthermore, the verification process introduces negligible compilation overhead and no runtime overhead, since all analyses are performed prior to deployment.

The proposed framework illustrates that workflow correctness can be treated as a compile-time concern rather than a runtime responsibility. By combining concepts from typestate systems, workflow verification, and Saga-based transactions, the approach provides stronger correctness guarantees for serverless orchestration while remaining practical for industrial cloud environments.

Several directions remain open for future work. First, we plan to investigate richer workflow type systems capable of expressing effect information and resource ownership. Second, we intend to extend the framework with support for parallel protocol verification and distributed workflow consistency analysis. Third, we plan to explore integration with workflow portability standards such as the CNCF Serverless Workflow specification. Finally, we aim to formally characterize the semantics of the DSL and establish correctness properties of the compilation process through machine-checked proofs.

The broader vision of this work is to bring the benefits of modern type systems to cloud orchestration. Just as type systems transformed software development by shifting error detection from runtime to compile time, we believe that workflow verification can play a similar role for serverless applications. The proposed framework represents an initial step toward a new generation of verified cloud workflows that combine practical deployment on industrial platforms with strong correctness guarantees.

Ultimately, we envision serverless workflows evolving from configuration-driven artifacts into statically verified software components whose correctness can be established before deployment. Such a shift would enable cloud applications to benefit from the same level of reliability, maintainability, and developer confidence that type systems have provided for modern programming languages.

References

1. van der Aalst, W.M.P.: The application of petri nets to workflow management. *Journal of Circuits, Systems and Computers* **8**(1), 21–66 (1998)
2. Amazon Web Services: Aws step functions developer guide. <https://docs.aws.amazon.com/step-functions/> (2025), accessed: 2026-06-17
3. Arjona, A., Garcia-Lopez, P., Sampe, J., Slominski, A., Villard, L.: Triggerflow: Trigger-based orchestration of serverless workflows. *Future Generation Computer Systems* (2021)

4. Baldini, I., Castro, P., Chang, K., Cheng, P., Fink, S., Ishakian, V., Mitchell, N., Muthusamy, V., Rabbah, R., Slominski, A., Suter, P.: Serverless computing: Current trends and open problems. In: *Research Advances in Cloud Computing*. pp. 1–20. Springer (2017)
5. Carbone, M., Honda, K., Yoshida, N.: Multiparty session types. *Journal of the ACM* **63**(1) (2016)
6. Castro, P., Ishakian, V., Muthusamy, V., Slominski, A.: The rise of serverless computing. *Communications of the ACM* **62**(12), 44–54 (2019)
7. Cloud Native Computing Foundation: Serverless workflow specification. <https://serverlessworkflow.io/> (2025), accessed: 2026-06-17
8. Decker, G., Weske, M.: Behavioral consistency for b2b process integration. *Information Systems* **33**(6), 547–566 (2008)
9. Fähndrich, M., DeLine, R.: Typestates for objects. In: *ECOOP*. pp. 465–490. Springer (2003)
10. Garcia-Molina, H., Salem, K.: Sagas. *Proceedings of the ACM SIGMOD International Conference on Management of Data* pp. 249–259 (1987). <https://doi.org/10.1145/38713.38742>
11. Hamadi, R., Benatallah, B.: A petri net-based model for web service composition. In: *ADC*. pp. 191–200 (2003)
12. Honda, K.: Types for dyadic interaction. In: *CONCUR*. pp. 509–523 (1993)
13. Hüttel, H., Lanese, I., Vasconcelos, V., Caires, L., Carbone, M., Deniélou, P.M., Mostrous, D., Padovani, L., Ravara, A., Tuosto, E., et al.: Foundations of session types and behavioural contracts. *ACM Computing Surveys* **49**(1) (2016)
14. Jonas, E., Schleier-Smith, J., Sreekanti, V., Tsai, C.C., Khandelwal, A., Pu, Q., Carreira, J., Krauth, K., Yadwadkar, N., Gonzalez, J., et al.: Cloud programming simplified: A berkeley view on serverless computing. *arXiv preprint arXiv:1902.03383* (2019)
15. Newman, S.: *Building microservices*. O’Reilly Media (2015)
16. Strom, R.E., Yemini, S.: Typestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering* **12**(1), 157–171 (1986). <https://doi.org/10.1109/TSE.1986.6312929>
17. Temporal Technologies: Temporal workflow orchestration. <https://temporal.io/> (2025), accessed: 2026-06-17
18. Uber Technologies: Cadence workflow orchestration. <https://cadenceworkflow.io/> (2025), accessed: 2026-06-17
19. Wen, J., Liu, Y.: A measurement study on serverless workflow services. In: *IEEE International Conference on Web Services (ICWS)* (2021)

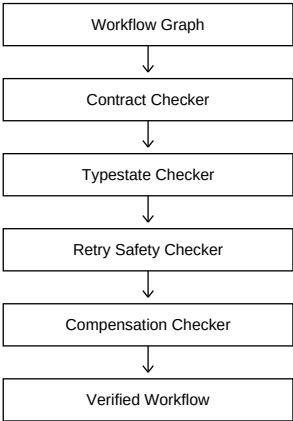


Fig. 4. Architecture of the static verification framework.

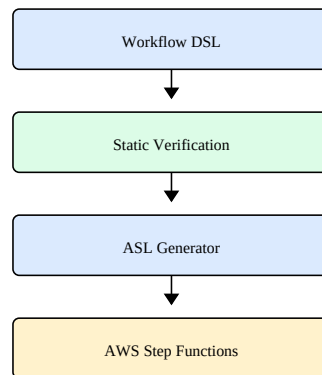


Fig. 5. Compilation architecture from the typed workflow DSL to AWS Step Functions.